

PatternRecognition– HW04: Information Gain & Mutual Information for Text Classification

Objective

The goal of this homework is to understand how feature selection based on Information Gain (IG) and Mutual Information (MI) can be used to choose the most informative words for a text classification task. After computing IG and MI for each word in the vocabulary, you will train a simple k-Nearest Neighbors (kNN) classifier on the selected features and predict the class of two unseen test documents.

Problem Setup

You are given a small collection of documents belonging to two classes:

- Class S — "Spam" (promotional / advertising messages).
- Class N — "Normal" (everyday text about animals).

Each document is represented as a bag of words. The training data has known class labels; the test data is unlabelled and must be predicted by your classifier.

TF: Term Frequencies and IDF: Inverse Document Frequencies. The dataset is shown in the table below.

Dataset Table

Data	Document	Words	Class
Training Data	d1	free, free, free, buy, discount, combo, pleasure	S
	d2	free, free, free, discount, pleasure, smile, smile, smile	S
	d3	cat, mouse	N
	d4	cat, cat, dog, dog, dog, dog	N
	d5	mouse	N
Test Data	d6	dog, cat, mouse, cat	???
	d7	free, free, smile	???

You are given the two classes, S and N, with related training data.

Tasks

You must complete the following tasks in order. Show all calculations.

Task 1 — Build the vocabulary and document-term matrix:

- List the full vocabulary V from the training set d1–d5 (lower-cased, no duplicates).

- For each training document, build a binary feature vector $x \in \{0,1\}^{|V|}$ where $x_t = 1$ if term t appears in the document and 0 otherwise. Show the matrix.
- Also build a TF (raw count) matrix and a TF-IDF matrix using $IDF(t) = \log(N / df(t))$ with $N = 5$ training documents. Show both matrices.

Task 2 — Information Gain for every term:

- Compute the entropy of the class variable on the training set: $H(C) = - \sum_c P(c) \log_2 P(c)$.
- For every term t in V compute the conditional entropy $H(C | t)$ using the binary feature x_t (term present vs. term absent), and the resulting Information Gain $IG(t) = H(C) - H(C | t)$.
- Rank all terms by IG in descending order. Present the result as a table.

Task 3 — Mutual Information for every term:

- For every term t compute the (pointwise-summed) Mutual Information between the binary feature X_t and the class C : $MI(X_t ; C) = \sum_{x \in \{0,1\}} \sum_{c \in \{S,N\}} P(x, c) \cdot \log_2 (P(x, c) / (P(x) P(c)))$.
- Use add-one (Laplace) smoothing on the joint counts to avoid $\log(0)$. Rank all terms by MI in descending order in a table.
- Compare the IG ranking and the MI ranking. Are they identical? Briefly explain why or why not for binary features.

Task 4 — Feature selection:

- Select the top- k most informative terms according to IG. Try $k = 2$, $k = 3$, and $k = 5$.
- For each k , build the reduced feature matrix for the training documents AND the two test documents (d6 and d7) using ONLY the selected terms.

Task 5 — kNN classification:

- Implement a simple kNN classifier from scratch (no sklearn for the classifier itself; you may use NumPy).
- Use cosine similarity as the similarity measure (so distance = $1 - \text{cosine similarity}$). Break ties by majority vote, then by smaller distance.
- For each k in $\{1, 3\}$ and each feature-set size from Task 4, predict the class of d6 and d7. Show the neighbours, their similarities, and the final predicted class for every (k , feature-set) combination.

Task 6 — Discussion:

- Did using IG/MI feature selection change the predictions compared to using the full vocabulary? Why?
- What happens if you use raw TF instead of binary features? What about TF-IDF?
- Discuss the limitations of IG / MI for this tiny dataset (data sparsity, smoothing choice, equal scores, near-zero counts).

Formulas (Quick Reference)

$$\begin{aligned}
 H(C) &= - \sum_c P(c) \log_2 P(c) \\
 H(C | X_t) &= \sum_x P(X_t = x) \cdot H(C | X_t = x) \\
 IG(t) &= H(C) - H(C | X_t) \\
 MI(X_t ; C) &= \sum_x \sum_c P(x, c) \cdot \log_2 [P(x, c) / (P(x) \cdot P(c))]
 \end{aligned}$$

$$\text{cos_sim}(u,v) = (u \cdot v) / (||u|| \cdot ||v||)$$

All probabilities are estimated from the 5 training documents only. Use base-2 logarithms throughout so that IG and MI are reported in bits.

Starter Code Skeleton (Python / NumPy)

You may use the following skeleton; complete the missing parts.

```
import numpy as np
from collections import Counter

# 1. Training and test data (bag of words)
train_docs = [
    ["free", "free", "free", "buy", "discount", "combo", "pleasure"], # d1 S
    ["free", "free", "free", "discount", "pleasure", "smile", "smile", "smile"], # d2 S
    ["cat", "mouse"], # d3 N
    ["cat", "cat", "dog", "dog", "dog", "dog"], # d4 N
    ["mouse"], # d5 N
]
train_labels = ["S", "S", "N", "N", "N"]

test_docs = [
    ["dog", "cat", "mouse", "cat"], # d6
    ["free", "free", "smile"], # d7
]

# 2. Vocabulary
vocab = sorted(set(w for doc in train_docs for w in doc))
word_to_idx = {w: i for i, w in enumerate(vocab)}

# 3. Binary, TF, and TF-IDF matrices
def to_binary(doc):
    v = np.zeros(len(vocab), dtype=int)
    for w in set(doc):
        if w in word_to_idx: v[word_to_idx[w]] = 1
    return v

def to_tf(doc):
    v = np.zeros(len(vocab), dtype=float)
    for w in doc:
        if w in word_to_idx: v[word_to_idx[w]] += 1
    return v

X_bin = np.array([to_binary(d) for d in train_docs])
X_tf = np.array([to_tf(d) for d in train_docs])

N = len(train_docs)
df = (X_bin > 0).sum(axis=0)
idf = np.log(N / np.maximum(df, 1))
X_tfidf = X_tf * idf
```

```

# 4. Information Gain -----
def entropy(labels):
    c = Counter(labels)
    total = sum(c.values())
    return -sum((n/total) * np.log2(n/total) for n in c.values() if n > 0)

def info_gain(term_idx, X_bin, labels):
    Hc = entropy(labels)
    present = X_bin[:, term_idx] == 1
    p1 = present.mean()
    p0 = 1 - p1
    H1 = entropy([labels[i] for i in range(len(labels)) if present[i]]) if p1 > 0
    else 0.0
    H0 = entropy([labels[i] for i in range(len(labels)) if not present[i]]) if p0 > 0
    else 0.0
    return Hc - (p1*H1 + p0*H0)

IG = {w: info_gain(i, X_bin, train_labels) for w, i in word_to_idx.items()}

# 5. Mutual Information (with add-one smoothing) -----
def mutual_info(term_idx, X_bin, labels, alpha=1.0):
    classes = sorted(set(labels))
    mi = 0.0
    n = len(labels) + 2*alpha # smoothed total
    for x in (0, 1):
        for c in classes:
            n_xc = sum(1 for i, l in enumerate(labels) if X_bin[i, term_idx] == x and
l == c) + alpha
            n_x = sum(1 for i in range(len(labels)) if X_bin[i, term_idx] == x) +
2*alpha
            n_c = sum(1 for l in labels if l == c) + 2*alpha
            p_xc = n_xc / n
            p_x = n_x / n
            p_c = n_c / n
            mi += p_xc * np.log2(p_xc / (p_x * p_c))
    return mi

MI = {w: mutual_info(i, X_bin, train_labels) for w, i in word_to_idx.items()}

# 6. Feature selection (top-k by IG) -----
def top_k_features(scores, k):
    return [w for w, _ in sorted(scores.items(), key=lambda kv: -kv[1])[:k]]

# 7. Simple kNN with cosine similarity -----
def cosine(u, v):
    nu, nv = np.linalg.norm(u), np.linalg.norm(v)
    return 0.0 if nu == 0 or nv == 0 else float(np.dot(u, v) / (nu*nv))

def knn_predict(X_tr, y_tr, x_te, k):
    sims = [cosine(x_te, x) for x in X_tr]
    order = np.argsort(sims)[::-1][:k]
    votes = Counter(y_tr[i] for i in order)
    return votes.most_common(1)[0][0], [(i, sims[i], y_tr[i]) for i in order]

```

```
# 8. Run experiments for k in {1,3} and feature-set sizes {2,3,5}
#   Print neighbours, similarities, and predicted classes for d6 and d7.
```

Submission

- A PDF (or Word) report containing all required tables, calculations, and discussion.
- A Jupyter / Colab notebook (.ipynb) with the full code for Tasks 1–5.
- A short conclusion summarising your final predictions for d6 and d7 for each (k, feature-set) combination.

Late submissions follow course policy.